

Entrada, salida y metodología de diseño

Objetivos

- Generar el entorno de desarrollo
- Reconocer la estructura básica de un programa en Java
- Comprender la diferencia entre sintaxis y semántica
- Entender el uso de plantillas sintácticas
- Ver la importancia de utilizar identificadores significativos
- Reconocer los tipos primitivos
- Conocer por qué hay diferentes tipos de datos numéricos con distintos rangos de valores
- Apreciar la diferencia entre una clase en sentido abstracto y como construcción de Java
- Ver la diferencia entre los objetos, en general, y su uso en Java
- Reconocer la diferencia entre los tipos carácter y cadena y cómo se relacionan
- Reconocer la diferencia entre constante y variable
- Saber qué es una asignación
- Comprender qué sucede cuando se invoca un método
- Conocer qué es una biblioteca
- Distinguir entre función y procedimiento
- Escribir sentencias sencillas de entrada y salida en Java

Contenido

El lenguaje Java	
Qué necesitamos	
Mi primer programa en Java	4
Entorno integrado de desarrollo	
Elementos básicos de un programa	

El lenguaje Java

A la hora de elegir un lenguaje de programación para aprender esta disciplina encontramos muchas y variadas opciones. Hoy en día se podría considerar que, como los lenguajes más populares son Python o JavaScript, estos deberían ser nuestros principales candidatos. De hecho, Python es reconocido como un lenguaje de programación ideal para principiantes por tener una sintaxis clara y legible...

Los siguientes enlaces lo son a diferentes listas elaboradas a partir de su popularidad: * [Índice TIOBE](#) * [Índice PYPL](#)

Como se puede ver, Java se encuentra en el Top 10 de la mayoría de clasificaciones. ¿Qué tiene Java de especial?

- **Tiene una sintaxis clara y legible:** Java es claro y fácil de entender...
- **Orientado a objetos:** Java es un lenguaje de programación orientado a objetos (OOP) donde los conceptos de objetos y clases son fundamentales.
- **Portabilidad:** Denominada *WORA* (Write Once, Run Anywhere).

Qué necesitamos

Para hacer un programa, en cualquier lenguaje de programación, necesitamos un editor de textos y un compilador o intérprete. En el caso de Java, necesitamos instalar el **JDK (Java Development Kit)**.

Tras la instalación, abre una consola o terminal y comprueba la versión ejecutando:

```
java -version
```

Configuración de variables de entorno

Si el terminal no reconoce el comando, sigue estos pasos según tu sistema operativo:

Windows

1. Abre las *Propiedades del sistema*.
2. Haz clic en *Variables de entorno*.
3. Selecciona la variable Path y haz clic en *Editar*.
4. Agrega la ruta al directorio binario del JDK al final de la lista.

Linux y MacOS

Edita el fichero `~/.bash_profile` o `.profile` y añade:

```
export PATH=/ruta/al/JDK/bin:$PATH
```

Ejecuta en tu terminal:

```
source ~/.bash_profile
```

Mi primer programa en Java

Abre tu editor de textos y escribe el siguiente código fuente:

```
public class MiPrimerPrograma {  
    public static void main ( String[] args ) {  
        System.out.println("Mi primer programa Java");  
    }  
}
```

Guárdalo con el nombre exacto `MiPrimerPrograma.java`. Desde la terminal, navega hasta la carpeta y compílalo:

```
javac MiPrimerPrograma.java
```

Para ejecutar el bytecode generado (`MiPrimerPrograma.class`), escribe:

```
java MiPrimerPrograma
```

Elementos básicos de un programa

Sintaxis y semántica

Un lenguaje de programación es un conjunto de reglas, símbolos y palabras especiales.

Sintaxis: Reglas formales que determinan la construcción de sentencias válidas en un lenguaje.

Semántica: El conjunto de reglas que dan el significado de una sentencia del lenguaje.

Metalinguaje: Lenguaje utilizado para describir las reglas sintácticas de otro lenguaje.

Plantillas sintácticas

Una plantilla sintáctica es un ejemplo genérico de la construcción lingüística que se está definiendo. Veamos una posible plantilla sintáctica para un programa en Java:

```
Programa:  
{ImportDeclaration}{ClassDeclaration}  
  
ClassDeclaration:  
{ClassModifier} class Identifier { {ClassBodyDeclaration} }
```

Los datos en cursiva indican que son *no terminales*. Si un símbolo no está en **bastardilla** y aparece en rojo, significa que es un *símbolo terminal*, como la palabra clave **class** y las llaves **{ }**.

Echemos un vistazo a cómo se define un identificador en el metalinguaje:

```
Identifier:  
IdentifierChars  
  
IdentifierChars:  
JavaLetter {JavaLetterOrDigit}  
  
JavaLetter:  
any Unicode character that is a "Java letter"
```

Paquete: Colección con nombre de clases de Java que puede ser utilizada por un programa.

Clase: Definición de un objeto o programa de Java.

Tipos de datos primitivos

Nombre del tipo	Tamaño usado (bits)	Total de números representables
byte	8 (1 octeto)	$2^8 = 256$
short	16 (2 octetos)	$2^{16} = 65.536$
int	32 (4 octetos)	$2^{32} = 4.294.967.296$
long	64 (8 octetos)	2^{64}